INF201 - Deliverable 3: Kachina Aschenberg Øwre & Seline Heggelund Knutsen

GitHub repository: https://github.com/kacoew/INF201-CAs (the code is in the folder `Deliverable 3`)

Task 2: Dice class for the battle simulator

This class represents a dice with a specified number of sides. It can be rolled to produce a random integer between 1 and the number of sides. The dice can also be called as a function to roll it, and it has a string representation for easy debugging and display. The Dice class is used in conjunction with the Weapon class to determine the damage dealt by weapons in the battle simulator.

In [6]:
```python
import random

class Dice:

    def __init__(self, sides: int = 6) -> None:  # __init__ + ValueError stops Dice(1) from being possible
        '''
        Initialises the Dice

        Args:
            sides(int): number of sides om the dice, default = 6
        '''
        if sides < 2:
            raise ValueError('Number of sides must be 2 or more') # a 1-sided dice makes no sense
        self.sides = sides

    def roll(self) -> int:
        '''Returns a random number between 1 and the number of sides'''
        return random.randint(1, self.sides)

    def __repr__(self) -> str:
        '''Representation of Dice object'''
        return f'Dice(sides={self.sides})'

    def __call__(self):
        '''Rolls the dice when the object is called as a function'''
        return self.roll()
```

In [7]:
```python
# Presentation of the Dice class: its attribute and methods
d6 = Dice()            # default: 6 sides
d20 = Dice(sides=20)

print("Attribute sides:", d6.sides, "and", d20.sides)
print("__repr__:", repr(d20))
print("roll():", d20.roll())
print("__call__:", d20())
```

```
rolls = [d6.roll() for _ in range(1000)]
print("1000 rolls of a d6 -> min:", min(rolls), "max:", max(rolls))

try:
    Dice(sides=1)
except ValueError as error:
    print("ValueError:", error)
```

```
Attribute sides: 6 and 20
__repr__: Dice(sides=20)
roll(): 17
__call__: 10
1000 rolls of a d6 -> min: 1 max: 6
ValueError: Number of sides must be 2 or more
```

Task 3: Weapon class

Items.py contains the Weapon class, which represents a weapon that can be equipped by a character. The damage dealt by the weapon is determined by rolling a dice, which is represented by the Dice class in dice.py. The Weapon class has methods to roll the damage and to provide a string representation of the weapon

In [8]:
```
from dice import Dice


class Weapon:
    """A weapon whose damage is decided by a dice roll."""

    def __init__(self, name: str, damage_dice: Dice) -> None:
        """
        Initialise the Weapon.

        Args:
            name (str): Name of the weapon, e.g. "Sword".
            damage_dice (Dice): The dice rolled to find the damage.
        """
        if not isinstance(damage_dice, Dice):
            raise TypeError("damage_dice must be an instance of Dice") # 
only accept a real Dice
        self.name = name
        self.damage_dice = damage_dice

    def roll_damage(self) -> int:
        """Roll the weapon's dice and return the damage."""
        return self.damage_dice.roll()

    def __repr__(self) -> str:
        """Representation like Weapon(name='Sword', 
damage_dice=Dice(sides=4))."""
        return f"Weapon(name={self.name!r}, damage_dice=
{self.damage_dice!r})"
```

```
if __name__ == "__main__":
    sword = Weapon("Sword", Dice(4))
    print(sword)
    rolls = [sword.roll_damage() for _ in range(1000)]
    print(min(rolls), max(rolls))

    try:
        Weapon("Stick", 6)
    except TypeError as error:
        print("TypeError:", error)
```

```
Weapon(name='Sword', damage_dice=Dice(sides=4))
1 4
TypeError: damage_dice must be an instance of Dice
```

Task 1, 3 and 4

Object-oriented game simulator - Based on game_simulator_v1.py by Turid Torheim, NMBU, 2026 Character.py is an updated version of the original, with a Weapon class added. The Weapon class is in items.py, and the Dice class is in dice.py. Task 1: Character class and Task 3: Weapon class are implemented in this file. The Character class can now take a Weapon object and use it in attacks. Task 1 says make a Character class with attributes for name, hp_max, damage, and methods for attack, take_damage, heal, and introduce. Task 3 adds the ability for a Character to have a Weapon, which uses a Dice object to determine damage. Task 4 adds two specialised subclasses:

- Warrior: reduces incoming damage using armor
- Mage: uses mana to cast magical attacks that deal double damage

In [9]:
```
from items import Weapon


class Character:
    """Represents a character in the RPG."""

    def __init__(self, name: str, hp_max: int = 100, damage: int = 10,    # 
__init__ stores all the info
                 weapon: Weapon | None = None) -> None:
        """
        Initialise the Character.

        Args:
            name (str): The name of the character.
            hp_max (int): Max hit points (default = 100).
            damage (int): Base damage when attacking without a weapon 
(default = 10).
            weapon (Weapon | None): Optional equipped weapon (default = 
None).
        """
        self.name = name
        self.hp_max = hp_max
        self._hp = hp_max  # private: only changed by take_damage() and
```

```
heal()
        self.damage = damage
        self.weapon = weapon

    @property
    def hp(self) -> int:  # Window to _hp
        """Current hit points (read-only)."""
        return self._hp

    @property
    def is_alive(self) -> bool:
        """True if the character has more than 0 HP (read-only)."""
        return self._hp > 0

    def take_damage(self, amount: int) -> None:  # ONLY WAY TO LOSE HP
        """Reduce own HP by ``amount``. HP never goes below 0."""
        if amount < 0:
            raise ValueError("Damage cannot be negative")
        self._hp = max(self._hp - amount, 0)

    def heal(self, amount: int) -> None:  # ONLY WAY TO GAIN HP
        """Increase own HP by ``amount``. HP never goes above hp_max."""
        if amount < 0:
            raise ValueError("Healing cannot be negative")
        if not self.is_alive:
            return  # a dead character cannot be healed (our choice)
        self._hp = min(self._hp + amount, self.hp_max)

    def roll_attack_damage(self) -> int:
        """Return how much damage this character deals right now."""
        if self.weapon is not None:
            return self.weapon.roll_damage()
        return self.damage

    def attack(self, target: "Character") -> int:
        """Attack ``target`` and return the damage dealt."""
        damage = self.roll_attack_damage()
        target.take_damage(damage)
        return damage

    def introduce(self) -> None:
        """Print a short introduction."""
        print(f"Hi, my name is {self.name}!")

    def __repr__(self) -> str:
        """Representation of the character object."""
        return f"<{type(self).__name__} name={self.name} hp={self.hp}
damage={self.damage}>"

    def __str__(self) -> str:
        """Readable description of the character."""
        return f"{type(self).__name__}: {self.name} with {self.hp} HP and
{self.damage} damage"


if __name__ == "__main__":
```

```
    from dice import Dice  # only needed for the tests below

    # Task 1 tests
    c = Character("Test")
    c.take_damage(150)
    print(c.hp, c.is_alive)              # expected: 0 False

    c = Character("Test")
    c.take_damage(30)
    c.heal(50)
    print(c.hp)                          # expected: 100 (not 120)

    link = Character("Link")
    sauron = Character("Sauron")

    link.attack(sauron)
    print(sauron.hp)                     # expected: 90

    try:
        sauron.hp = 999
    except AttributeError as error:
        print("AttributeError:", error)  # expected: hp cannot be set from
outside

    try:
        c.take_damage(-5)
    except ValueError as error:
        print("ValueError:", error)      # expected: Damage cannot be
negative

    # Task 3 tests
    dummy = Character("Dummy", hp_max=1000)
    a = Character("A")
    print("Damage on dummy (no weapon):  ", [a.attack(dummy) for _ in
range(5)])   # always 10
    zelda = Character("Zelda", weapon=Weapon("Sword", Dice(20)))
    print("Damage on dummy with weapon:", [zelda.attack(dummy) for _ in
range(5)])    # random 1–20



# Task 4


class Warrior(Character):
    '''Defensive character that reduces incoming damage using armor'''
    def __init__(self, name, hp_max: int=100, damage: int=10, weapon:
Weapon | None = None, armor: int = 5) -> None:
        '''
        Initialise the Warrior

            Args:
                name (str): The name of the warrior
                hp_max (int): Maximum hit points (default = 100)
                damage (int): Base attack damage (default = 10)
                weapon (Weapon | None): Optional equipped weapon
```

```
                armor (int): Damage reduction applied to incoming attacks
(default = 5)
        '''
        super().__init__(name, hp_max, damage, weapon)      # Character
sets up the basics. Lets you use super. to use Character`s functions and
attributes
        self.armor = armor      # Warrior adds what`s new

    def take_damage(self, amount: int) -> None:     # OVERRIDE
        '''
        Reduce incoming damage by armor before applying it

        Args:
            amount (int): Incoming damage
        '''
        reduce_damage = max(0, amount - self.armor)     # take armor off
first
        super().take_damage(reduce_damage)      # Character does the rest


class Mage(Character):
    '''Magic-based character that uses mana to cast powerful attacks'''
    def __init__(self, name, hp_max: int = 100, damage: int = 10,
                weapon: Weapon | None = None, mana: int = 30, mana_max: int
= 30) -> None:
        '''
        Initialise the Mage

        Args:
            name (str): The name of the mage
            hp_max (int): Maximum hit points (default = 100)
            damage (int): Base attack damage (default = 10)
            weapon (Weapon | None): Optional equipped weapon
            mana (int): Current mana (default = 30)
            mana_max (int): Maximum mana (default = 30)
        '''
        super().__init__(name, hp_max, damage, weapon)      # no need to
repeat the same stuff as Character
        self.mana = mana
        self.mana_max = mana_max

    def attack(self, target: 'Character') -> int: # OVERRIDE
        '''
        Attack a target

        If the mage has at least 10 mana, a magical attack is used
        The attack consumes 10 mana and deals double base damage

        If the mage has less than 10 mana, a normal attack is used
        and the mage restores 5 mana

        Args:
            target (Character): The target being attacked

        Returns:
            int: The damage dealt
```

```
        '''
        if self.mana >= 10: # magical attack if enough mana
            self.mana -= 10 # consume 10 mana

            damage = 2 * self.damage # double base damage
            target.take_damage(damage)

            return damage

        else: # not enough mana, normal attack and restore 5 mana
            self.mana = min(self.mana + 5, self.mana_max)

            return super().attack(target) # use Character's attack method
for normal attack



# Task 4 tests


warrior = Warrior("Aragorn")
warrior.take_damage(10)
print(warrior.hp)            # 95

mage = Mage("Gandalf")
orc = Character("Orc")

print(mage.mana)             # 30
print(mage.attack(orc))      # 20
print(mage.mana)             # 20

mage.mana = 5
print(mage.attack(orc))      # normal attack
print(mage.mana)             # 10
```

```
0 False
100
90
AttributeError: property 'hp' of 'Character' object has no setter
ValueError: Damage cannot be negative
Damage on dummy (no weapon):   [10, 10, 10, 10, 10]
Damage on dummy with weapon: [9, 19, 12, 14, 3]
95
30
20
20
10
10
```

Task 5: Running battles

Create a Battle class to manage turn-based combat between two Character objects.

In [10]:
```
from character import Character, Warrior, Mage
from items import Weapon
```

```python
from dice import Dice


class Battle:
    """Manages a turn-based battle between two characters."""

    def __init__(self, hero: Character, enemy: Character, max_rounds: int =
100) -> None:      # Battle has two characters
        """
        Initialise the Battle.

        Args:
            hero (Character): The player's character.
            enemy (Character): The opposing character.
            max_rounds (int): Stop after this many rounds (default = 100).
        """
        self.hero = hero
        self.enemy = enemy
        self.max_rounds = max_rounds
        self.round = 0

    def play_round(self) -> None:
        """Hero attacks enemy; the enemy hits back only if still alive."""
        self.round += 1
        hero_damage = self.hero.attack(self.enemy)      # 1. hero always
hits
        print(f"{self.hero.name} deals {hero_damage} damage.")
        print(f"{self.enemy.name}: {self.enemy.hp} HP")

        if self.enemy.is_alive:     # 2. enemy hits back only if alive
            enemy_damage = self.enemy.attack(self.hero)
            print(f"{self.enemy.name} deals {enemy_damage} damage.")
            print(f"{self.hero.name}: {self.hero.hp} HP")

        print()

    def simulate_battle(self) -> Character | None:
        """
        Run battle rounds until one character dies, or max_rounds is
reached.

        Returns:
            Character | None: The winning character, or None if it is a
draw.
        """
        while self.hero.is_alive and self.enemy.is_alive and self.round <
self.max_rounds:       # 3. keep going while both live
            self.play_round()

        if not self.enemy.is_alive:
            return self.hero        # 4. return the winner
        if not self.hero.is_alive:
            return self.enemy

        return None  # both still alive after max_rounds
```

```python
if __name__ == "__main__":

    # Test 1: Warrior vs Character
    hero = Warrior("Aragorn")
    enemy = Character("Orc")

    battle = Battle(hero, enemy)

    winner = battle.simulate_battle()

    print(f"\nWinner: {winner.name}")
    print(f"{hero.name}: {hero.hp} HP")
    print(f"{enemy.name}: {enemy.hp} HP")

    print("\n" + "-" * 30 + "\n")

    # Test 2: Warrior vs Mage
    hero = Warrior("Aragorn")
    enemy = Mage("Saruman")

    battle = Battle(hero, enemy)

    winner = battle.simulate_battle()

    print(f"\nWinner: {winner.name}")
    print(f"{hero.name}: {hero.hp} HP")
    print(f"{enemy.name}: {enemy.hp} HP")

    print("\n" + "-" * 30 + "\n")

    # Test 3, battle with weapons
    sword = Weapon("Anduril", Dice(20))
    staff = Weapon("Wizard Staff", Dice(20))

    hero = Warrior("Aragorn", weapon=sword)
    enemy = Mage("Saruman", weapon=staff)

    battle = Battle(hero, enemy)
    winner = battle.simulate_battle()

    print(f"\nWinner: {winner.name}")
    print(f"{hero.name}: {hero.hp} HP")
    print(f"{enemy.name}: {enemy.hp} HP")
```

```
Aragorn deals 10 damage.
Orc: 90 HP
Orc deals 10 damage.
Aragorn: 95 HP

Aragorn deals 10 damage.
Orc: 80 HP
Orc deals 10 damage.
Aragorn: 90 HP

Aragorn deals 10 damage.
Orc: 70 HP
Orc deals 10 damage.
Aragorn: 85 HP

Aragorn deals 10 damage.
Orc: 60 HP
Orc deals 10 damage.
Aragorn: 80 HP

Aragorn deals 10 damage.
Orc: 50 HP
Orc deals 10 damage.
Aragorn: 75 HP

Aragorn deals 10 damage.
Orc: 40 HP
Orc deals 10 damage.
Aragorn: 70 HP

Aragorn deals 10 damage.
Orc: 30 HP
Orc deals 10 damage.
Aragorn: 65 HP

Aragorn deals 10 damage.
Orc: 20 HP
Orc deals 10 damage.
Aragorn: 60 HP

Aragorn deals 10 damage.
Orc: 10 HP
Orc deals 10 damage.
Aragorn: 55 HP

Aragorn deals 10 damage.
Orc: 0 HP


Winner: Aragorn
Aragorn: 55 HP
Orc: 0 HP

------------------------------

Aragorn deals 10 damage.
```

```
Saruman: 90 HP
Saruman deals 20 damage.
Aragorn: 85 HP

Aragorn deals 10 damage.
Saruman: 80 HP
Saruman deals 20 damage.
Aragorn: 70 HP

Aragorn deals 10 damage.
Saruman: 70 HP
Saruman deals 20 damage.
Aragorn: 55 HP

Aragorn deals 10 damage.
Saruman: 60 HP
Saruman deals 10 damage.
Aragorn: 50 HP

Aragorn deals 10 damage.
Saruman: 50 HP
Saruman deals 10 damage.
Aragorn: 45 HP

Aragorn deals 10 damage.
Saruman: 40 HP
Saruman deals 20 damage.
Aragorn: 30 HP

Aragorn deals 10 damage.
Saruman: 30 HP
Saruman deals 10 damage.
Aragorn: 25 HP

Aragorn deals 10 damage.
Saruman: 20 HP
Saruman deals 10 damage.
Aragorn: 20 HP

Aragorn deals 10 damage.
Saruman: 10 HP
Saruman deals 20 damage.
Aragorn: 5 HP

Aragorn deals 10 damage.
Saruman: 0 HP


Winner: Aragorn
Aragorn: 5 HP
Saruman: 0 HP

------------------------------

Aragorn deals 5 damage.
Saruman: 95 HP
```

```
Saruman deals 20 damage.
Aragorn: 85 HP

Aragorn deals 8 damage.
Saruman: 87 HP
Saruman deals 20 damage.
Aragorn: 70 HP

Aragorn deals 2 damage.
Saruman: 85 HP
Saruman deals 20 damage.
Aragorn: 55 HP

Aragorn deals 17 damage.
Saruman: 68 HP
Saruman deals 19 damage.
Aragorn: 41 HP

Aragorn deals 6 damage.
Saruman: 62 HP
Saruman deals 7 damage.
Aragorn: 39 HP

Aragorn deals 16 damage.
Saruman: 46 HP
Saruman deals 20 damage.
Aragorn: 24 HP

Aragorn deals 10 damage.
Saruman: 36 HP
Saruman deals 9 damage.
Aragorn: 20 HP

Aragorn deals 15 damage.
Saruman: 21 HP
Saruman deals 13 damage.
Aragorn: 12 HP

Aragorn deals 8 damage.
Saruman: 13 HP
Saruman deals 20 damage.
Aragorn: 0 HP


Winner: Saruman
Aragorn: 0 HP
Saruman: 13 HP
```

Summary of Gen AI use

Seline: I used Microsoft Copilot as a guidance tool throughout this assignment. The AI helped me understand the task requirements, explained object-oriented programming concepts such as classes, inheritance, and methods, and provided suggestions for implementing the required functionality. It was also used for debugging, code

documentation, and understanding how different files and classes interact. The solutions were implemented by me, while the AI served as a learning resource and guide throughout the development process.

Kachina: I used Claude as a tutor and code reviewer throughout this assignment. It gave me a plan with code templates, and explained concepts such as encapsulation, properties, composition and super(), as well as how the classes and files work together. I wrote the code for Task 1 and the Weapon class myself. Claude reviewed it and helped me debug errors, and for one bug I used the corrected class it provided. At the end, it made some small changes I asked for and approved, such as adding a round limit to Battle. It also helped me draft this summary. I have tested all the submitted code myself.